PPM: Automated Generation of Diverse Programming Problems for Benchmarking Code Generation Models

> #Benchmark #Evaluation

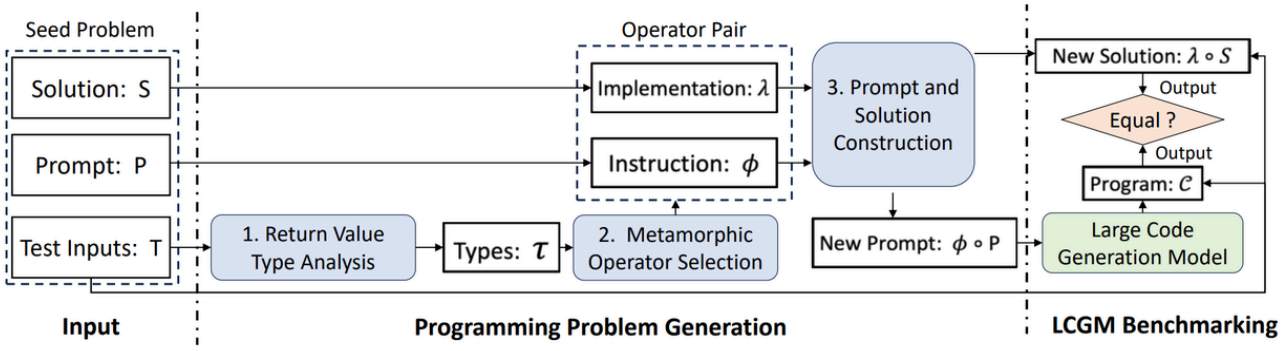

Fig. 2. Design overview of PPM.

- What problems does this framework solve?
  - When evaluating coding abilities of LLMs, it is tough and expensive to create tests by humans.
  - Once the tests are uploaded to the Internet, they will be crawled and used in LLM training, causing data contamination.
  - Automatically added noises, such as modifying the description of the problems, will not change the nature of the problem (the model can still answer the question by reciting what it has memorized).
  - These automatically added noises may also include spell errors and grammar errors, which are unlikely to be seen or be problems during actual use. The tests will be absurd.
- What does this framework do?
  - Intuition: Concatinate two problems together to create a brand new question.
  - Step 1: The framework will analyze the output type of the seeding questions as the most questions are in weakly typed languages such as python. It will run the groundtruth codes, collect the answer, and analyze the type. If the results are iteratives, then the framework will also, layer by layer, analyze the types.
  - Step 2: For each seeding question, the framework will choose a **Metamorphic Operator**, normally containing two parts: NL description and a piece of code.
    - Description: What does this operator do.
    - Code: Normally an easy lambda expression.
    - To avoid contamination, the lambda expression contains a random variable `offset` to ensure the uniqueness and randomness.
  - Step 3: The selected description will be appended to the original description, and the new groundtruth output will be generated by feeding the original codes' output to the lambda function, creating a new description. Meanwhile, the lambda function will be added to original codes.

- Experiment Results
  - These datasets exhibit extremely high semantic diversity: Compared to baseline methods, 100% of the programming problems generated by PPM require code implementations that are completely different from the original problems to be solved (with a DiffImp score of 100%).
  - Highly natural and realistic: As measured by perplexity and human blind evaluation scores, PPM-generated problems significantly outperform traditional random substitution methods in both linguistic fluency and logical soundness, and rarely trigger syntax warnings in IDEs.
  - Effectively exposes model flaws: New problems generated by PPM immediately and significantly reduce the accuracy of all tested LCGMs, causing the Pass@1 metric to plummet by over 90% in some cases. This demonstrates that existing models often merely "memorize" problems found online rather than truly possessing generalizable programming reasoning capabilities.
- Work Cited
  - Paper
  - NotebookLM
  - Chat History